

# GPUMODE Lecture Study Notes: Lecture 44

## NVIDIA Profiling

### Core Problem the Lecture Addresses

This lecture explains how to profile CUDA kernels using NVIDIA developer tools, especially **Nsight Systems** and **Nsight Compute**. The central engineering problem is:

How do we move from vague observations such as “my GPU code is slow” to a precise hardware-level explanation of which resource, pipeline, memory path, instruction, or source line is limiting performance?

The lecture emphasizes that profiling is not just looking at one metric. It is a workflow:

1. Use **Nsight Systems** to understand the full application timeline.
2. Identify whether the GPU is actually on the critical path.
3. Find expensive kernels, idle regions, synchronization, copies, and CPU–GPU gaps.
4. Use **Nsight Compute** to deeply inspect one kernel or a small range of kernels.
5. Read hardware counters, source correlation, assembly correlation, memory traffic, scheduler stalls, occupancy, roofline models, and rule-based recommendations.
6. Modify the kernel, profile again, and compare reports.

The key point is that GPU optimization requires a connection between three views:

Algorithmic expectation  $\longleftrightarrow$  Source code  $\longleftrightarrow$  Hardware counters.

A good profiler helps verify whether the hardware is doing what the programmer

expected. For example, if a vector add should read two vectors and write one vector, the profiler should show roughly:

$$\text{Read traffic} = 2N \cdot \text{sizeof}(\text{dtype}), \quad \text{Write traffic} = N \cdot \text{sizeof}(\text{dtype}).$$

If the measured traffic is much larger, then the implementation has unexpected memory behavior, poor access patterns, redundant loads, excessive cache traffic, or compiler-generated instructions that were not intended.

## Required Background

### CUDA Execution Model

CUDA kernels execute over a grid of thread blocks. Each block contains threads. Threads are grouped into warps of 32 threads on NVIDIA GPUs.

$$\text{Grid} \rightarrow \text{Thread blocks} \rightarrow \text{Warps} \rightarrow \text{Threads}.$$

A typical one-dimensional indexing pattern is:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

This maps each CUDA thread to one logical element of an array.

### Streaming Multiprocessors and Warp Schedulers

A GPU consists of multiple streaming multiprocessors, abbreviated as SMs. Each SM contains execution pipelines, registers, shared memory, L1 cache, warp schedulers, and load-store units.

In the lecture, Nsight Compute is described as exposing metrics close to the hardware. A useful mental model is:

Kernel  $\rightarrow$  thread blocks scheduled onto SMs  $\rightarrow$  warps issued by warp schedulers  $\rightarrow$  instructions executed by p

On many NVIDIA architectures, each SM contains multiple warp schedulers. The lecture highlights the common model where each SM has four warp schedulers, and each warp scheduler can issue approximately one instruction per cycle under ideal conditions.

$$\text{Maximum issue rate per SM} \approx 4 \text{ warp instructions per cycle}.$$

This does not mean every cycle reaches this rate. Warps may be stalled on memory dependencies, barriers, instruction dependencies, or pipeline availability.

## Memory Hierarchy

The memory hierarchy discussed in the lecture includes:

- **Registers:** private to each thread, fastest storage.
- **Shared memory:** explicitly managed memory shared by threads in a block.
- **L1 cache:** close to each SM.
- **L2 cache:** shared across the GPU.
- **HBM or device DRAM:** high-bandwidth global memory.
- **PCIe or NVLink:** host-device or device-device interconnects.

A load from global memory may pass through L1 and L2 before reaching DRAM. If it misses in both caches, the latency can be hundreds of cycles. Nsight Compute can attribute stalls to this behavior.

## Arithmetic Intensity and Roofline Analysis

Arithmetic intensity is:

$$I = \frac{\text{FLOPs executed}}{\text{Bytes moved}}.$$

The roofline model relates arithmetic intensity to attainable performance:

$$P_{\text{attainable}} = \min(P_{\text{peak}}, I \cdot B_{\text{peak}}),$$

where:

- $P_{\text{attainable}}$  is the achievable throughput.
- $P_{\text{peak}}$  is the peak compute throughput.
- $B_{\text{peak}}$  is the peak memory bandwidth.
- $I$  is arithmetic intensity.

If a kernel lies near the sloped memory-bandwidth line, it is memory-bound. If it lies near a horizontal compute ceiling, it is compute-bound.

## Main Concepts

### Nsight Systems Versus Nsight Compute

The lecture clearly separates the role of the two profiling tools.

## **Nsight Systems**

Nsight Systems is a high-level platform profiler. It answers questions such as:

- Is the GPU busy or idle?
- Is the CPU launching work efficiently?
- Are memory copies overlapping with computation?
- Are multiple GPUs active?
- Are there synchronization gaps?
- Which kernels dominate the timeline?
- Is the application CPU-bound, GPU-bound, copy-bound, or synchronization-bound?

It is timeline-oriented. It captures CPU threads, CUDA API calls, GPU kernels, memory transfers, synchronization, process activity, and some device-wide GPU metrics.

Nsight Systems is intended to have low overhead and preserve the original application behavior as much as possible.

## **Nsight Compute**

Nsight Compute is a low-level kernel profiler. It answers questions such as:

- Which kernel instruction is expensive?
- Which memory path is saturated?
- What is the L1 or L2 hit rate?
- Which warp stall reason dominates?
- Which source line maps to which SASS instructions?
- Is the kernel limited by DRAM, shared memory, L2, tensor cores, FP64, ALU, or another pipeline?
- What changed after a code modification?

Nsight Compute is not primarily meant to profile an entire application with thousands of kernels. It is meant for deep analysis of one kernel or a carefully selected kernel range.

## **Recommended Profiling Workflow**

The recommended workflow is:

1. Run the application in Nsight Systems.
2. Identify whether the GPU is on the critical path.

3. Identify the kernel or small range of kernels worth optimizing.
4. Launch Nsight Compute on that kernel.
5. Inspect high-level summary metrics first.
6. Use the rule system to find likely bottlenecks.
7. Drill into memory workload, scheduler statistics, warp stall states, instruction mix, occupancy, and source correlation.
8. Modify code.
9. Re-profile.
10. Use Nsight Compute comparison mode to compare reports.
11. Return to Nsight Systems to see whether the whole application improved.

The final step matters because optimizing one kernel can shift the application bottleneck. For example, after a kernel becomes faster, memory copies may no longer overlap, or a different kernel may become the critical path.

## Profiler Interception Model

Nsight Compute launches the target application through the profiler:

```
ncu ./my_application arg1 arg2
```

The profiler sits between the application and the CUDA driver. It intercepts CUDA driver interactions and can observe kernel launches without requiring source-code modification.

The application does not need to be rewritten, but optional changes improve profiling quality:

- Add NVTX annotations to mark high-level regions.
- Compile CUDA code with `--lineinfo` to enable source-to-assembly correlation.
- In Numba, enable line information through JIT options.
- Use memory access flags when possible so the profiler can avoid unnecessary save-and-restore work.

A typical CUDA compilation command is:

```
nvcc -O3 --lineinfo kernel.cu -o kernel
```

The `--lineinfo` flag does not disable optimization. It asks the compiler to emit mapping information from high-level source lines to low-level generated assembly.

## NVTX Instrumentation

NVTX allows the user to annotate application regions. These annotations appear in NVIDIA profiling tools.

A common use is to restrict Nsight Compute profiling to kernels launched inside a named range.

```
#include <nvtx3/nvToolsExt.h>

void run_step() {
    nvtxRangePushA("forward_pass");
    launch_my_kernel();
    nvtxRangePop();
}
```

This is useful when an application launches many kernels and only a specific phase is relevant.

## Kernel Replay and Multi-Pass Collection

A major concept in the lecture is that Nsight Compute often cannot collect all metrics in a single kernel execution. Hardware observability is finite. There are limited counters, limited buffer bandwidth, and limited profiling hardware.

Therefore, Nsight Compute may replay a kernel multiple times.

A simplified replay sequence is:

1. Save accessible GPU state.
2. Optionally lock GPU clocks.
3. Optionally clear caches.
4. Configure a subset of metrics.
5. Launch the kernel once.
6. Collect profiling data.
7. Restore modified state.
8. Repeat for the next metric subset.

If a report requires  $M$  metrics but only  $K$  compatible metrics can be collected per pass, then the approximate number of passes is:

$$N_{\text{passes}} \approx \left\lceil \frac{M}{K} \right\rceil.$$

This is not exact because metric compatibility is not uniform. Some metrics can be collected together, while others require separate configurations.

## Why Replay Exists

Replay is not mainly for averaging. It exists because different metrics require different hardware counter configurations.

For example, suppose a hypothetical cache metric required separate hit and miss collection:

$$\text{Hit rate} = \frac{\text{Hits}}{\text{Hits} + \text{Misses}}.$$

If hits and misses were measured in inconsistent runs, the computed hit rate could be meaningless. This is why the profiler tries to make each replay pass begin from a consistent state.

## State Save and Restore

Before replaying a kernel, Nsight Compute saves accessible memory that the kernel may read or write. After a pass, it restores modified state so the next pass observes the same input.

The save location may be:

- GPU memory, if available.
- Host memory, if GPU memory is insufficient.
- Disk, in worst-case scenarios.

The fastest case is a device-to-device copy. The slowest case is spilling saved state to disk.

The profiler can optimize restore by identifying which memory regions changed. If input buffers are read-only, they do not need to be restored after each pass.

## Cache Clearing

Nsight Compute may clear caches before each pass to ensure consistent replay. The lecture stresses that “caches” here means more than L2. It can include:

- L1 cache.
- L2 cache.
- Constant cache.
- Instruction cache.
- Other architecture-specific caches.

This matters because metrics collected over multiple passes should describe the same kernel behavior. If one pass begins with cold caches and another with warm caches, aggregate derived metrics may be misleading.

## Single-Pass Versus Multi-Pass Profiling

If only a small number of compatible metrics are requested, profiling may require one pass. But full reports may require many passes.

The trade-off is:

More metrics  $\Rightarrow$  more passes  $\Rightarrow$  longer profiling wall-clock time.

However, the measured kernel execution inside each replay pass is intended to remain faithful to the unprofiled kernel execution. The overhead mostly surrounds the kernel launch, not the kernel instruction stream itself.

## Hardware-Level Explanation

### Speed of Light Metrics

Nsight Compute uses “Speed of Light” sections to summarize how close a kernel is to hardware peak.

The major high-level metrics include:

- Compute throughput as a percentage of peak.
- Memory throughput as a percentage of peak.
- L1 throughput.
- L2 throughput.
- DRAM throughput.
- Pipeline utilization.
- Scheduler utilization.

A vector add example in the lecture shows low compute utilization and high DRAM utilization. That is exactly what should be expected:

$$C_i = A_i + B_i.$$

Each element does one floating-point add but moves multiple values through memory.

For FP32:

Bytes per element = 4 bytes for  $A_i$  + 4 bytes for  $B_i$  + 4 bytes for  $C_i$  = 12 bytes.

The arithmetic intensity is:



$$I_{\text{vecadd}} = \frac{1 \text{ FLOP}}{12 \text{ bytes}} \approx 0.083 \text{ FLOPs per byte.}$$

This is extremely low, so the kernel is expected to be memory-bound.

## Memory Workload Analysis

The memory workload chart shows traffic through the memory hierarchy. For vector addition, the expected behavior is:

$$\text{Loads : Stores} = 2 : 1.$$

If each vector is 64 MiB, then expected traffic is approximately:

$$\text{Load traffic} = 128 \text{ MiB}, \quad \text{Store traffic} = 64 \text{ MiB}.$$

The lecture notes that a 33% L2 hit rate can appear because stores may be counted as hits while loads miss. For a streaming vector add that touches each input once, L1 and L2 reuse is not expected to help much.

The correct interpretation is not simply “low cache hit rate is bad.” Instead:

A low cache hit rate is only bad if the algorithm expected temporal or spatial reuse. A streaming kernel that touches each element once naturally stresses DRAM.

## Warp Scheduler Statistics

A warp scheduler can issue instructions from eligible warps. Nsight Compute distinguishes several concepts:

- **Active warps:** warps resident on the SM.
- **Eligible warps:** active warps ready to issue an instruction.
- **Stalled warps:** active warps not ready to issue.
- **Selected warp:** the warp chosen for issue in a given cycle.

A useful inequality is:

$$\text{eligible warps} \leq \text{active warps} \leq \text{maximum resident warps}.$$

If there is a large gap between active and eligible warps, many warps are stalled. If eligible warps per scheduler are far below one, the scheduler frequently has no ready warp to issue.

The lecture describes a useful visual interpretation:

- A cliff between theoretical and active warps suggests occupancy constraints.
- A cliff between active and eligible warps suggests stalls.
- Eligible warps near or above one per scheduler can be enough to saturate issue, depending on the instruction mix.

## Warp Stall Reasons

Nsight Compute reports stall reasons such as:

- Long scoreboard.
- Short scoreboard.
- Wait.
- Barrier.
- Not selected.
- Branch-related stalls.
- Dependency-related stalls.

### Long Scoreboard

Long scoreboard often means a warp is waiting for a prior memory operation, commonly a global memory load.

For example:

$$r_4 \leftarrow A_i, \quad r_7 \leftarrow B_i, \quad r_8 \leftarrow r_4 + r_7.$$

The add cannot execute until both loads complete. If the loads miss L1 and L2, the warp may wait hundreds of cycles.

In source-correlated assembly, this appears as the arithmetic instruction accumulating stall samples because the arithmetic instruction is waiting on load-produced registers.

### Short Scoreboard

Short scoreboard is often associated with shorter-latency dependencies, including shared-memory-related waits or other near-SM dependencies. It can also arise in more complex instruction dependency situations.

## Instruction Mix

Nsight Compute shows the SASS instruction histogram. In vector add, the expected dominant instructions include:

- Global loads.
- Floating-point add.
- Global store.
- Integer arithmetic for address calculation.
- Special register reads for block and thread indices.

A simplified expected instruction pattern is:

```
S2R      // read special register, such as threadIdx.x
IMAD     // compute global index
LDG      // load A[i]
LDG      // load B[i]
FADD     // add A[i] and B[i]
STG      // store C[i]
```

The instruction mix helps answer:

- Did the compiler generate the instruction type I expected?
- Am I accidentally using FP64 instead of FP32?
- Are tensor cores actually used?
- Are there many integer instructions for address arithmetic?
- Are there local memory instructions due to spilling?

## Mathematical Reasoning

### Vector Add Arithmetic Intensity

For the kernel

$$C_i = A_i + B_i,$$

assuming FP32 arrays:

$$\text{FLOPs per element} = 1.$$

The bytes moved per element are:

$$\text{Bytes per element} = 4 + 4 + 4 = 12.$$

Therefore:

$$I = \frac{1}{12} \approx 0.083 \text{ FLOPs per byte.}$$

If DRAM bandwidth is  $B_{\text{DRAM}}$ , the roofline-predicted upper bound is:

$$P_{\text{vecadd}} \leq I \cdot B_{\text{DRAM}}.$$

For example, if:

$$B_{\text{DRAM}} = 900 \text{ GB/s,}$$

then:

$$P_{\text{vecadd}} \leq 0.083 \cdot 900 \approx 75 \text{ GFLOP/s.}$$

This is small relative to GPU peak compute. Therefore, optimizing math instructions will not help much unless memory traffic is reduced or arithmetic intensity increases.

## Memory Traffic Sanity Check

For  $N$  FP32 elements:

$$\text{Total bytes} = N \cdot 12.$$

For  $N = 16,777,216$ , each vector has:

$$N \cdot 4 = 67,108,864 \text{ bytes} \approx 64 \text{ MiB.}$$

Then:

$$\text{Input reads} = 2 \cdot 64 = 128 \text{ MiB,}$$

and:

$$\text{Output writes} = 64 \text{ MiB.}$$

This is the kind of napkin math the lecture recommends doing before opening the profiler. The profiler then confirms or disproves the mental model.

## Warp Issue Efficiency

Let  $W_a$  be active warps per scheduler and  $W_e$  be eligible warps per scheduler. The scheduler can issue at most one instruction per cycle. A rough issue utilization model is:

$$U_{\text{issue}} \approx \min(W_e, 1).$$

This is not a complete performance model because instruction pipeline availability, instruction latency, dependencies, and dual-issue behavior may matter. But it is a useful first approximation:

$$W_e \ll 1 \quad \Rightarrow \quad \text{scheduler often idle.}$$

$$W_e \geq 1 \quad \Rightarrow \quad \text{scheduler usually has a warp ready.}$$

## Replay Pass Cost Model

Let:

$$T_k = \text{kernel runtime,}$$

$$T_s = \text{state save time,}$$

$$T_r = \text{state restore time per pass,}$$

$$T_c = \text{counter configuration and collection overhead per pass,}$$

$$P = \text{number of replay passes.}$$

A rough profiling wall-clock time model is:

$$T_{\text{profile}} \approx T_s + P \cdot (T_c + T_k + T_r).$$

This explains why profiling a long-running kernel with many passes can be expensive, even if the per-pass kernel measurement is accurate.

## Pipeline Bottleneck Reasoning

If a kernel has pipeline utilization values:

$$U_{\text{FP64}} = 84\%, \quad U_{\text{ALU}} = 33\%, \quad U_{\text{DRAM}} = 3\%,$$

then the bottleneck is likely FP64 compute, not memory. If the source code did not intend FP64 math, then the optimization target is precision control.

If replacing FP64 operations with FP32 reduces runtime from  $T_{\text{old}}$  to  $T_{\text{new}}$ , the speedup is:

$$S = \frac{T_{\text{old}}}{T_{\text{new}}}.$$

The percent runtime reduction is:

$$R = \left(1 - \frac{T_{\text{new}}}{T_{\text{old}}}\right) \cdot 100\%.$$

In the lecture example, Nsight Compute estimated roughly an 82% improvement from removing FP64 operations, and the measured improvement was close to that estimate.

## Code Walkthrough

### Minimal Vector Add Kernel

```
__global__ void vector_add(const float* A,
                           const float* B,
                           float* C,
                           int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < N) {
        C[i] = A[i] + B[i];
    }
}
```

### Indexing Logic

The line

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

maps each CUDA thread to one array element. At the SASS level, this usually involves reading special registers and doing integer arithmetic.

The hardware behavior is:

thread  $\rightarrow$  computes  $i \rightarrow$  loads  $A_i \rightarrow$  loads  $B_i \rightarrow$  adds  $\rightarrow$  stores  $C_i$ .

### Memory Coalescing

For consecutive threads in a warp:

$$i_t = i_0 + t, \quad t \in \{0, 1, \dots, 31\}.$$

The addresses are contiguous:

$$\text{addr}(A_{i_t}) = \text{base}(A) + 4(i_0 + t).$$

This is coalesced for FP32. A warp loads a contiguous segment of memory. The same applies to  $B$  and  $C$ .

### Expected Bottleneck

The kernel has very low arithmetic intensity. It should be DRAM bandwidth-bound for large  $N$ . Therefore, a good profile should show:

- High DRAM throughput.
- Low compute throughput.
- Long scoreboard stalls.
- Load-to-add dependencies.
- Approximately two loads per store.

### Nsight Compute Command-Line Profiling

```
ncu --set full --kernel-name regex:vector_add ./vector_add
```

The intent is:

- `ncu`: launch under Nsight Compute.
- `--set full`: collect a broad report.
- `--kernel-name`: restrict profiling to matching kernels.
- `./vector_add`: target executable.

## Using NVTX to Select a Kernel Region

```
#include <nvtx3/nvToolsExt.h>

void training_iteration() {
    nvtxRangePushA("attention_forward");
    launch_attention_kernel();
    nvtxRangePop();
}
```

Then Nsight Compute can be configured to profile kernels launched inside this named range. This avoids collecting reports for unrelated kernels.

## Python and Numba Precision Pitfall

The lecture gives an example of a Numba image-processing kernel where Python floating-point literals caused unintended FP64 operations. This is common because Python floating-point literals are double precision by default.

Problematic style:

```
@cuda.jit(lineinfo=True)
def sharpen_kernel(inp, out, n):
    i = cuda.grid(1)
    if i < n:
        x = inp[i] / 255.0
        y = x * 0.5
        out[i] = y
```

The constants 255.0 and 0.5 may encourage double-precision operations depending on compiler inference and context.

Safer style:

```
@cuda.jit(lineinfo=True)
def sharpen_kernel(inp, out, n):
    i = cuda.grid(1)
    if i < n:
        x = inp[i] / np.float32(255.0)
        y = x * np.float32(0.5)
        out[i] = y
```

The hardware implication is significant on consumer GPUs because FP64 throughput is often much lower than FP32 throughput.

A simplified ratio is:

$$\frac{P_{\text{FP32}}}{P_{\text{FP64}}} \gg 1.$$



On some consumer GPUs this ratio can be very large, such as 64:1. Therefore, accidental FP64 can dominate runtime.

## Tensor Core and TMA Conceptual Kernel

The lecture includes a Blackwell-style example involving tensor memory, tensor cores, and TMA. The simplified operation is:

$$C = AB.$$

The conceptual flow is:

1. Use TMA to copy tiles of  $A$  and  $B$  from global memory to shared memory.
2. Use tensor core MMA instructions to compute partial products.
3. Accumulate results in tensor memory or an architecture-specific accumulator structure.
4. Load results back into registers.
5. Store results to global memory.

A simplified pseudocode sketch is:

```
__global__ void tensor_core_kernel(const half* A,
                                   const half* B,
                                   half* C) {
    // 1. Prepare descriptors for tiled memory movement.
    // 2. Use asynchronous TMA copy from global memory to shared memory.
    // 3. Wait on a barrier until the copy is visible.
    // 4. Execute tensor core MMA operations.
    // 5. Move accumulator fragments to registers.
    // 6. Store results to global memory.
}
```

The profiler's memory workload chart can reveal:

- Global-to-shared traffic through TMA.
- Shared-memory reads feeding tensor core operations.
- Tensor memory activity on supported architectures.
- Register-file reads and writes.
- Final global stores.

# Nsight Compute Report Structure

## Summary Page

The summary page shows one row per profiled kernel or range. Typical fields include:

- Kernel name.
- Duration.
- Compute throughput.
- Memory throughput.
- Register count.
- Grid dimensions.
- Block dimensions.
- Rule-based warnings or recommendations.

This page is a triage view. It tells the engineer where to inspect next.

## Session Page

The session page records launch information:

- Application path.
- Command-line arguments.
- Profiling options.
- Device attributes.
- Collection configuration.

Device attributes matter because the same kernel can behave differently on different GPUs.

## Raw Metrics Page

The raw page exposes collected metric values. This is useful for:

- Exporting metrics.
- Building custom analysis.
- Checking exact counter names.
- Comparing against scripts or notebooks.

Nsight Compute reports can also be accessed from Python through a public report interface based on protocol buffers.

## Details Page

The details page is organized into sections. Each section contains:

- A description.
- Important metrics.
- Charts or tables.
- Rule outputs.
- Links to deeper sections or documentation.

Important sections include:

- Speed of Light.
- Memory Workload Analysis.
- Scheduler Statistics.
- Warp State Statistics.
- Instruction Statistics.
- Launch Statistics.
- Occupancy.
- Source Counters.

## Source Page

The source page correlates high-level code with generated SASS assembly. It can show:

- Source lines.
- Assembly instructions.
- Execution counts.
- Stall samples.
- Memory access counts.
- Register dependencies.
- Compiler annotations.
- Excessive memory sectors.
- Local memory usage and spilling annotations.

The lecture warns not to start with the source page blindly. A better workflow is:

Details page → identify bottleneck → Source page → find responsible line or instruction.

## Performance Intuition

### Start With an Expected Model

Before reading profiler metrics, estimate what should happen.

For a kernel, ask:

- How many bytes should be read?
- How many bytes should be written?
- How many FLOPs should occur?
- Should tensor cores be used?
- Should shared memory be used?
- Should L1 or L2 cache help?
- Should the kernel be memory-bound or compute-bound?

Then compare the measured profile to that expectation.

### Do Not Treat Every Warning as Equally Important

Nsight Compute may show multiple recommendations. Some are not worth fixing first.

For example, an image kernel may have imperfect memory coalescing, but if FP64 pipeline utilization dominates runtime, then coalescing is not the first bottleneck.

The optimization priority should follow expected impact:

$$\text{Priority} \approx \text{bottleneck severity} \times \text{fixability} \times \text{application-level importance}.$$

### Memory-Bound Kernel Intuition

A memory-bound kernel often has:

- High DRAM percentage of peak.
- Low compute utilization.
- Low arithmetic intensity.
- Long scoreboard stalls.

- Streaming access patterns.
- Little cache reuse.

Good optimizations include:

- Reduce bytes moved.
- Fuse kernels to avoid intermediate writes.
- Improve coalescing.
- Improve data reuse with tiling.
- Use shared memory when reuse exists.
- Use vectorized memory operations when appropriate.
- Avoid unnecessary precision or layout conversions.

## Compute-Bound Kernel Intuition

A compute-bound kernel often has:

- High utilization of one or more compute pipelines.
- Lower memory throughput.
- High arithmetic intensity.
- Stall reasons related to execution dependencies or pipeline pressure.
- Instruction mix dominated by arithmetic operations.

Good optimizations include:

- Use tensor cores if mathematically valid.
- Avoid accidental FP64.
- Use faster approximations if precision allows.
- Improve instruction-level parallelism.
- Balance pipeline usage.
- Reduce dependency chains.
- Avoid unnecessary conversions.

## Occupancy Is Not the Same as Performance

High occupancy helps hide latency, but it is not the final goal. A kernel can have high occupancy and still be slow if every warp is waiting on DRAM. A kernel can have moderate occupancy and still be fast if it has enough eligible warps and strong data reuse.

A better mental model is:

Performance  $\neq$  Occupancy alone.

Instead:

Performance  $\approx f(\text{active warps, eligible warps, instruction mix, memory traffic, pipeline balance, latency hiding})$

## Why Comparison Mode Matters

After changing code, compare old and new reports. Do not rely only on runtime.

Nsight Compute comparison mode can show:

- Runtime reduction.
- Bottleneck shifts.
- Pipeline utilization changes.
- Memory traffic changes.
- Stall reason changes.
- Source-line metric changes.

A successful optimization usually changes the bottleneck. For example, removing FP64 operations may shift the main limiter from FP64 pipeline pressure to ALU or memory behavior.

## Common Misconceptions

### Misconception: Nsight Compute Should Be Used First

Nsight Compute is not the first tool for whole-application diagnosis. Use Nsight Systems first to identify whether GPU kernels are the issue.

### Misconception: More Metrics Always Means Better Profiling

More metrics may require more replay passes. Collect what you need. A full report is useful for deep diagnosis, but targeted metric sets are better for fast iteration.

### Misconception: Cache Hit Rate Should Always Be High

A streaming kernel may naturally have low cache hit rate. The real question is whether the algorithm expected reuse.

### **Misconception: High Occupancy Guarantees High Performance**

Occupancy only describes how many warps can reside on an SM. It does not guarantee those warps are eligible to issue instructions.

### **Misconception: Source Code Shows What the GPU Executes**

The compiler may transform code heavily. It may unroll loops, introduce local memory, change instruction selection, or emit FP64 operations because of literal types. Use source-to-assembly correlation.

### **Misconception: CUDA Cores Are the Only Useful Hardware Model**

CUDA core counts are useful at a high abstraction level, but profiler metrics expose lower-level pipeline behavior. The profiler may describe pipelines differently from whitepaper-level CUDA core descriptions.

### **Misconception: All Stalls Are Equally Bad**

A stall reason matters only in relation to total samples, issue utilization, and bottleneck severity. Optimizing a rare stall may have no measurable effect.

## **Practical Takeaways**

1. Start in Nsight Systems, then move to Nsight Compute.
2. Profile one kernel or one meaningful range, not the whole application by default.
3. Compile with `--lineinfo` for CUDA source correlation.
4. Use NVTX ranges to isolate important phases.
5. Do napkin math before opening profiler reports.
6. Check whether memory traffic matches algorithmic expectation.
7. Use Speed of Light metrics to classify memory-bound versus compute-bound behavior.
8. Use memory workload charts to locate the hot memory path.
9. Use scheduler statistics to see whether warps are eligible to issue.
10. Use warp stall reasons to understand why eligible warps are missing.
11. Use instruction statistics to verify instruction mix.

12. Use source correlation only after identifying the bottleneck.
13. Watch for accidental FP64, especially in Python and Numba.
14. Compare reports before and after optimization.
15. Return to Nsight Systems to verify application-level improvement.

## Project or Experiment Ideas

### Experiment 1: Vector Add Roofline

Implement vector addition and profile it.

Measure:

- DRAM throughput.
- L1 and L2 hit rates.
- Long scoreboard stalls.
- Arithmetic intensity.
- Roofline position.

Expected result:

$$I \approx 0.083 \text{ FLOPs per byte,}$$

so the kernel should be memory-bound.

### Experiment 2: Accidental FP64 in Python

Write a Numba CUDA kernel using constants such as `255.0`. Profile it. Then replace constants with `np.float32(...)` and profile again.

Compare:

- FP64 pipeline utilization.
- Kernel duration.
- Eligible warps per scheduler.
- Instruction mix.
- Source-level stall samples.



### Experiment 3: Coalesced Versus Strided Loads

Create two kernels:

```
C[i] = A[i];  
C[i] = A[i * stride];
```

Vary **stride**. Profile memory sectors, L1/L2 traffic, and DRAM throughput.

Expected result:

larger stride  $\Rightarrow$  less coalescing  $\Rightarrow$  more excessive memory sectors.

### Experiment 4: Shared Memory Tiling

Implement naive matrix multiplication and tiled matrix multiplication.

Measure:

- Global memory traffic.
- Shared memory traffic.
- Arithmetic intensity.
- Occupancy.
- Stall reasons.
- Runtime.

Expected improvement comes from reuse:

reuse  $\Rightarrow$  fewer global bytes per FLOP  $\Rightarrow$  higher arithmetic intensity.

### Experiment 5: Report Comparison Workflow

Create an intentionally slow kernel. Profile it. Make one optimization. Profile again and use Nsight Compute comparison mode.

Track whether the bottleneck shifted.

## Final Mental Model

Nsight Systems tells you **where time goes across the application**. Nsight Compute tells you **why one kernel behaves the way it does on the GPU**.

The best profiling workflow is not metric hunting. It is a disciplined loop:

predict  $\rightarrow$  measure  $\rightarrow$  explain  $\rightarrow$  change  $\rightarrow$  compare  $\rightarrow$  validate globally.

For every kernel, build an expectation:

expected bytes,      expected FLOPs,      expected instruction types,      expected memory path.

Then use the profiler to test that expectation. If measured traffic, stalls, pipelines, or instructions disagree with your model, that disagreement is the optimization clue.

### **Visual Concept**

Draw a two-stage workflow. On the left, show Nsight Systems as a horizontal application timeline with CPU threads, CUDA API calls, memory copies, and GPU kernels. On the right, zoom into one kernel with Nsight Compute, showing SMs, warp schedulers, memory hierarchy, SASS instructions, source correlation, roofline, and bottleneck rules.